Лабораторна робота №9

Архітектурна модель системи та метрики якості моделей

Сервіс замовлення їжі з доставкою

Виконав: Некрасов Артур Дмитрович

Група: ІПЗ-23-3

2026

1 Контекст проєкту і сценарії якості

Для запуску сервісу доставки їжі в одному місті обрано модульний моноліт. Він дає змогу двом розробникам випускати один серверний застосунок і змінювати оплату без перероблення доставки. Рішення спирається на чотири сценарії якості; їхні цільові значення ще потрібно перевірити на реалізації.

Пілот обслуговують двоє розробників, які також відповідають за супровід. Стартовий бюджет обмежено одним орендованим VPS і однією керованою базою PostgreSQL; окремого бюджету на брокер та кластер немає. Орієнтир піку — 200 замовлень за годину та 40 кур'єрів на зміні. Найвищий пріоритет — швидко вносити зміни за відгуками перших клієнтів.

Клієнт замовляє та оплачує їжу; працівник закладу приймає замовлення; кур'єр отримує пропозицію й підтверджує доставку. Оплата, карти й push-сповіщення виконуються через зовнішні системи. Зв'язок цих ролей і модулів із попередніми роботами наведено в розділі 4.

Сценарій і атрибут	Джерело і подразник	Артефакт і середовище	Відповідь	Міра відповіді
С1 Модифікованість	**Джерело:** власник продукту **Подразник:** новий провайдер оплати	**Артефакт:** модуль «Оплата» **Середовище:** звичайна розробка, чинний контракт API	Додати адаптер провайдера і контрактний тест.	≤ 2 робочі дні; змінено один модуль; зовнішній API сумісний.
С2 Розгортуваність	**Джерело:** розробник **Подразник:** випуск нової серверної версії	**Артефакт:** серверний застосунок **Середовище:** робочий день, один VPS і керована БД	Один конвеєр виконує міграцію, запуск і перевірку готовності.	≤ 15 хв від запуску конвеєра до готовності; один серверний артефакт.
С3 Доступність	**Джерело:** збій процесу **Подразник:** аварійна зупинка сервера	**Артефакт:** сервер і збережені замовлення **Середовище:** пік, VPS та БД доступні	Супервізор перезапускає процес; сервер відновлює обробку з БД.	Відновлення ≤ 120 с; втрачено 0 підтверджених замовлень.
С4 Продуктивність	**Джерело:** 200 паралельних клієнтів **Подразник:** читають каталог	**Артефакт:** API каталогу **Середовище:** пік, прогрітий кеш, тест 5 хв	Сервер віддає каталог із кешу; зміни меню інвалідовують запис.	p95 ≤ 1 с; частка помилок < 1%.

С1 оцінюють за журналом змін і контрактним тестом; С2 — за журналом конвеєра. Для С3 вимірюють час до успішного запиту після зупинки процесу і звіряють ідентифікатори підтверджених замовлень. Для С4 p95 — час, у який вкладаються 95% виміряних відповідей.

2 Архітектурні альтернативи та ADR

Шкала: 2 — добре підтримує сценарій у заданому контексті; 1 — потрібні додаткові заходи; 0 — суперечить стартовим обмеженням. Ваги C1, C2, C3, C4: 3, 3, 2, 1. Бал — сума оцінок, помножених на ваги (максимум 18).

Стиль	C1 Зміни	C2 Випуск	C3 Відновлення	C4 Каталог	Бал
A1 Шаруватий моноліт	1 — логіка оплати розподіляється між шарами.	2 — один збірний артефакт.	1 — спільний процес потребує перезапуску.	2 — локальні виклики, можливий кеш.	13
A2 Модульний моноліт	2 — адаптери й тести в «Оплаті».	2 — один конвеєр для сервера.	1 — супервізор і фіксація запису в БД.	2 — локальні виклики, кеш каталогу.	16
A3 Мікросервіси	2 — платіжний сервіс змінюється окремо.	0 — кілька випусків, складніший супровід.	1 — ізоляція збою, але без резервного вузла.	1 — незалежне масштабування, мережеві витрати.	9

ADR 001 Стиль серверного застосунку

Статус: Прийнято для архітектурної моделі пілота.

Контекст і варіанти: Двоє розробників, один VPS, до 200 замовлень за годину. Вирішальними є C1 і C2; оцінено A1, A2 та A3.

Рішення: A2: Java 21 і Spring Boot, один серверний процес із модулями «Замовлення», «Оплата», «Доставка», «Сповіщення», «Спільне». Модулі викликають публічні інтерфейси або слухають внутрішні події. PostgreSQL має окремі схеми для модулів, що зберігають дані; прямий доступ до чужих таблиць заборонено.

Позитивні наслідки: Один серверний артефакт спрощує випуск; платіжні адаптери мають чітке місце; межі модулів дають підставу згодом виділити сервіс без зміни його бізнес-правил.

Негативні наслідки: Збій процесу зачіпає всі серверні функції; масштабують увесь сервер, навіть якщо вузьке місце лише в доставці; межі пакетів потрібно контролювати архітектурними тестами.

Умова перегляду: Переглянути рішення, якщо пік перевищує 2000 замовлень за годину протягом трьох піків поспіль, з’являються три незалежні команди або C3 поширюється на відмову всього VPS. Спочатку виміряти вузьке місце й порівняти репліки моноліту з виділенням модуля.

3 Архітектурна модель C4

3.1 Діаграма контексту

Рівень 1 показує межу сервісу та шість зв’язків. Три ролі працюють із сервісом, а шлюз оплати, карти й push-провайдер виконують зовнішні функції. Контейнери та внутрішні модулі на цьому рівні не деталізуються.

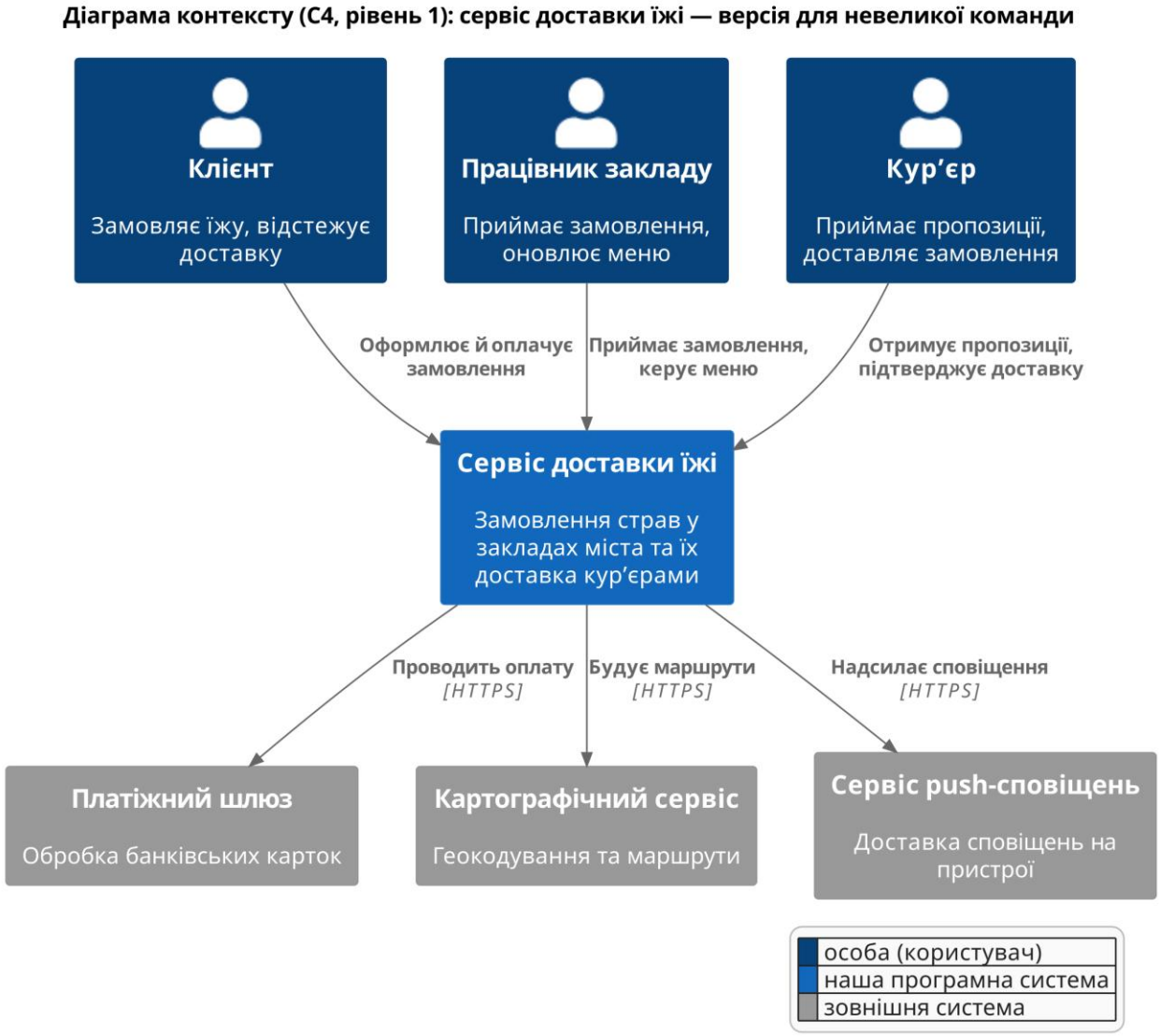


Рисунок 1 — Контекст сервісу доставки їжі. Три особи, одна система, три зовнішні системи.

3.2 Діаграма контейнерів

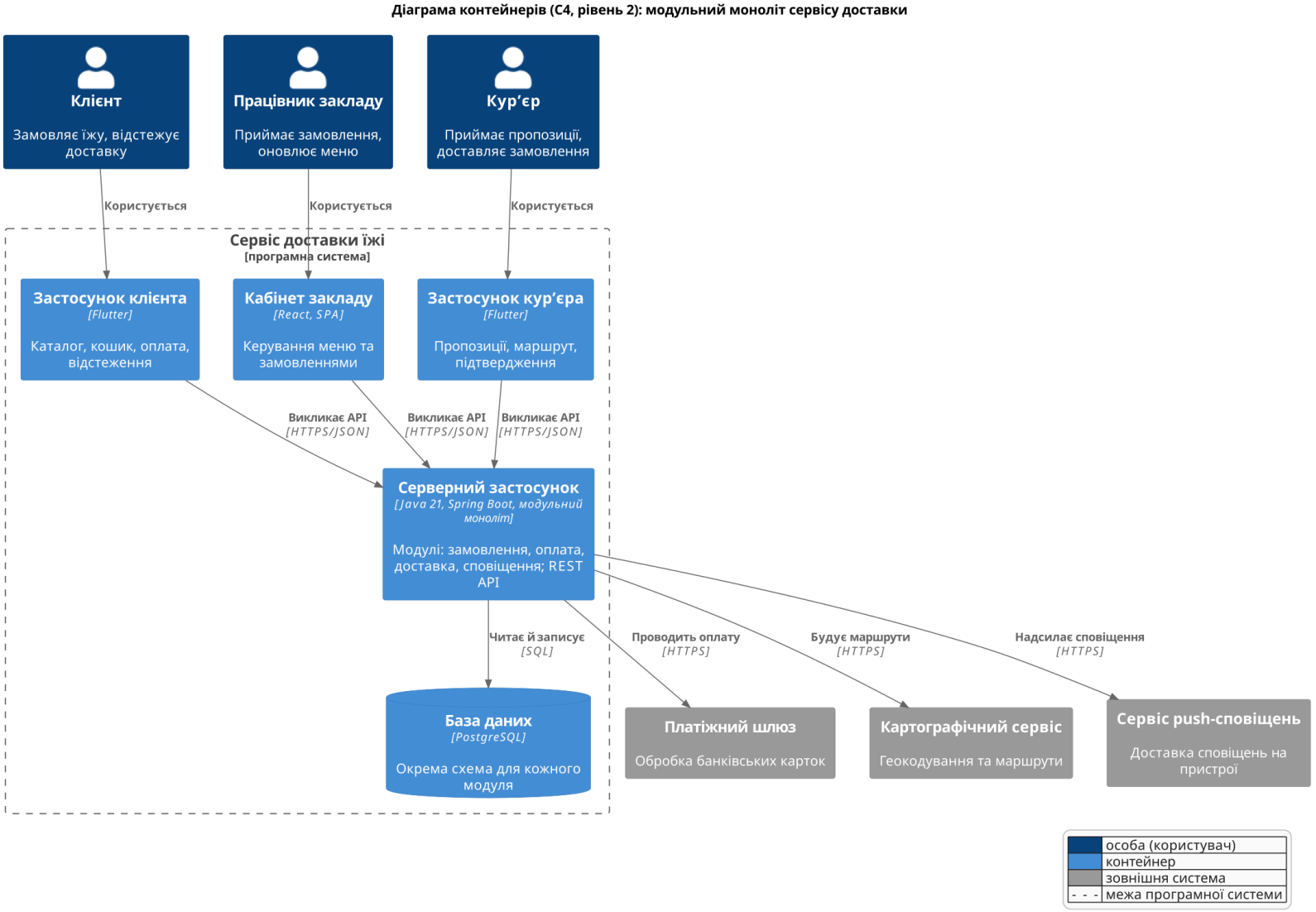


Рисунок 2 — П'ять контейнерів. Клієнти звертаються до API через HTTPS/JSON, сервер — до PostgreSQL через SQL.

Кабінет закладу — React SPA; застосунки клієнта й кур'єра — Flutter. Сервер звертається до шлюзу, карт і push через HTTPS. Позначка SQL описує запити; з'єднання з PostgreSQL передбачено через JDBC і TCP/TLS. Код — додаток В.

3.3 Модулі серверного застосунку

Рівень 3 деталізує один серверний контейнер. Стрілка A → B означає, що A залежить від інтерфейсу або типу події B. Наприклад, «Доставка» слухає подію «ЗамовленняОплачено» з «Замовлень».

Діаграма компонентів (C4, рівень 3): модулі серверного застосунку та їхні залежності

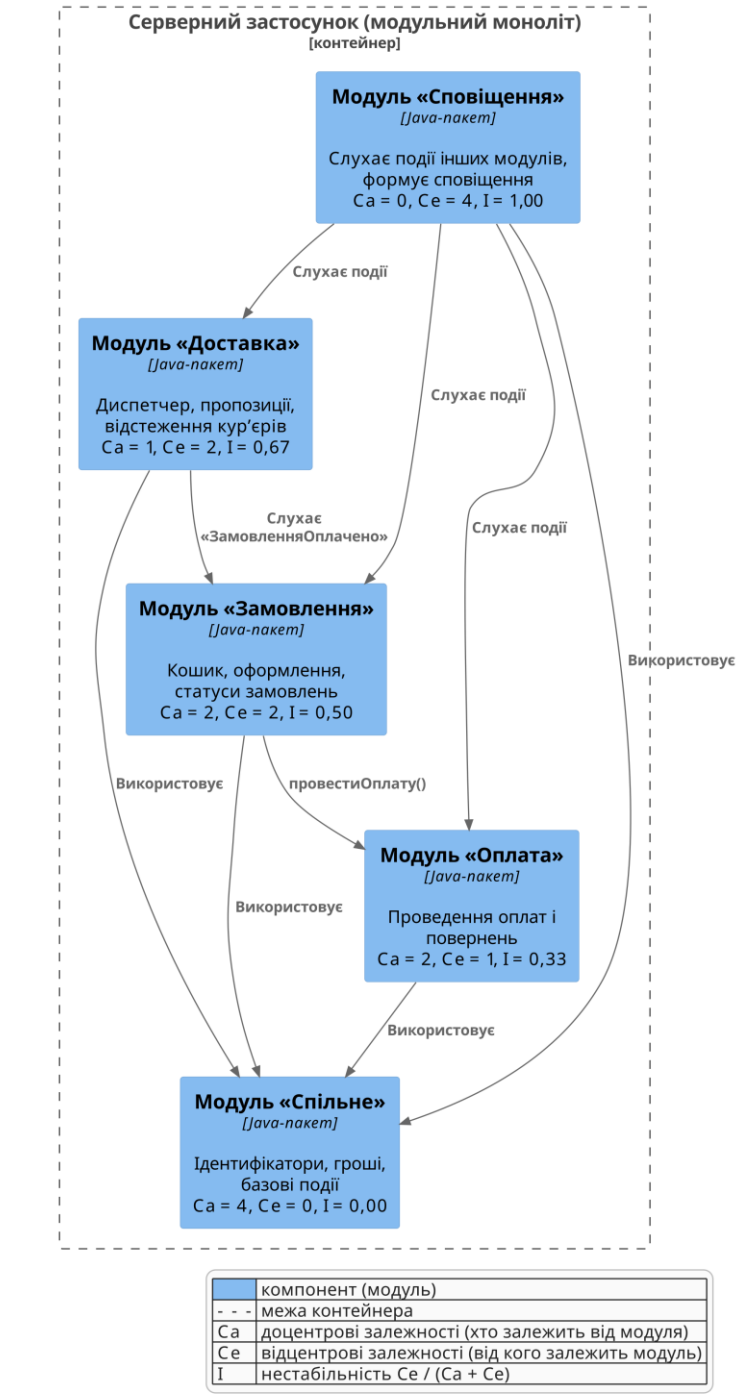


Рисунок 3 — П'ять модулів і дев'ять прямих залежностей. Метрики на схемі звірено з розрахунком у розділі 7.

Код — додаток Г. Рисунки 1–3 відтворюють надані SVG; кількість елементів і зв'язків звірено з кодом PlantUML.

4 Зв’язок з попередніми лабораторними

Архітектуру зіставлено з еталонними описами Лаб. 1 і Лаб. 3 у методичці [1, §3.8]. Таблиця показує, що саме перенесено до C4, які елементи змінилися після ADR 001 і що слід оновити в попередніх моделях.

Відповідник у попередній моделі	Елемент Лаб. 9	Пояснення і потрібна зміна
Лаб. 1: клієнт, зареєстрований клієнт, ресторан-партнер, кур’єр	Клієнт, працівник закладу, кур’єр на C4 L1	Ролі збережено. Для закладу названо людину, що працює з кабінетом; види клієнтів лишаються в Use Case.
Лаб. 1: адміністратор	Окрема роль і контейнер відсутні	Відкрита розбіжність. Уточнити обов’язковість сценарію; якщо потрібен, додати адміністратора й інтерфейс з окремою роллю доступу.
Лаб. 1: платіжна система; Лаб. 3: платіжний шлюз	Шлюз оплати, карти, push на C4 L1 і L2	Шлюз збережено. Карти й push виділено як зовнішні залежності; додати вторинних акторів до відповідних Use Case.
Лаб. 3: компоненти замовлень, оплати, доставки й сповіщень	Чотири бізнес-модулі одного контейнера	Відповідальності збережено; виклики між серверними частинами стають локальними або подійними за ADR 001.
Лаб. 3: API Gateway	REST API серверного контейнера	Окремий шлюз більше не потрібний для маршрутизації між сервісами. Автентифікацію й перевірку ролей виконує веб-рівень сервера.
Лаб. 3: order-service.jar, payment-service.jar, notification-service.jar	Один Spring Boot артефакт на VPS	Оновити діаграму розгортання: один процес, PostgreSQL — окремий керований вузол. Це підтримує C2.
Лаб. 3: вебклієнт і food-delivery-app.apk	Два Flutter застосунки та React кабінет	Розділено клієнтські можливості за ролями. Усі використовують HTTPS/JSON, окремі серверні процеси для них не потрібні.
Лаб. 3: окремого спільного компонента немає	Пакет «Спільне» на C4 L3	Додано місце для стабільних ідентифікаторів, грошей і базових типів подій; бізнес-логіка лишається у своїх модулях.

Диспетчеризація у Лаб 7 і Лаб 8

Лаб. 7. Схема «Диспетчерська» у Z і специфікація VDM [2] задають чергу, призначення й активні пропозиції. Ці обов’язки віднесено до модуля «Доставка». Перед видачею пропозиції він має перевіряти стан «Очікує», рейтинг ≥ 35 і відсутність іншої активної пропозиції для замовлення.

Лаб. 8. Alloy-модель [3] розрізняє безпеку (не дублювати пропозицію) і живучість (не допустити нескінченного циклу відмов). Подія оплати запускає постановку в чергу; обмеження повторних відмов і таймер залишаються окремим бізнес-правилом для уточнення.

Лаб. 2 задає класи та зв’язки для CK; Лаб. 6 — повідомлення для RFC. Лаб. 4 і 5 дають графи діяльності та станів для V(G). Розрахунки в розділах 5–6 відтворюють еталонний інвентар із [1, §3.9–3.10].

5 Метрики класів CK і RFC

WMC — кількість оголошених методів (вага 1, успадковані не додаються). DIT — кількість кроків узагальнення до кореня; реалізація інтерфейсу не впливає. NOC — прямі підкласи або реалізації інтерфейсу.

CBO — кількість різних сусідів за асоціацією, агрегацією, композицією і типами параметрів або результатів, якщо їх показано; зв'язок рахуємо з обох боків. Узагальнення й реалізації виключено. LCOM не обчислено: діаграма не показує доступ методів до атрибутів.

Клас або інтерфейс	WMC	DIT	NOC	CBO	Сусіди за CBO
Особа	2	0	2	0	—
Клієнт	2	1	0	1	Замовлення
Кур'єр	2	1	0	1	Замовлення
Оцінюваний	2	0	2	0	—
Заклад	2	0	0	1	Страва
Страва	2	0	0	2	Заклад, ПозиціяЗамовлення
Замовлення	2	0	0	4	Клієнт, Кур'єр, Оплата, ПозиціяЗамовлення
ПозиціяЗамовлення	2	0	0	2	Замовлення, Страва
Оплата	2	0	0	1	Замовлення
Сума / максимум	**18 / 2**	**— / 1**	**— / 2**	**12 / 4**	**6 різних пар**

Ручна перевірка. У «Замовлення» чотири сусіди: Клієнт, Кур'єр, Оплата, ПозиціяЗамовлення. У «Кур'єр» один сусід — Замовлення; перехід до «Особа» дає DIT = 1 і не збільшує CBO. Сума CBO = 12 = 2 × 6. Для інтерфейсу «Оцінюваний» еталон дає дві реалізації.

Відгук класу за послідовністю Лаб 6

RFC = M + R. M — різні власні методи, R — різні безпосередні зовнішні операції сценарію A. Повторні виклики не додаємо, транзитивні виклики не розгортаємо. Створення пропозиції у сценарії є окремою операцією.

Клас	Власні методи M	Зовнішні виклики R	RFC
Диспетчер	4: призначитиКур'єра, відібратиКандидатів, прийняти, відхилити	5: Пропозиція.«create»; Кур'єр.надіслатиПропозицію; Кур'єр.прийнятиЗамовлення; Заклад.повідомитиПроКур'єра; ДодатокКлієнта.надіслатиДаніКур'єра	9
Кур'єр	3: прийнятиЗамовлення, підтвердитиДоставку, надіслатиПропозицію	2: Диспетчер.прийняти; Диспетчер.відхилити	5

Однаковий WMC = 2 показує рівень деталізації Лаб. 2. У Лаб. 6 з'являється ще одна операція кур'єра, а «Диспетчер» (RFC = 9) узагалі відсутній у класовій моделі. «Замовлення» має найбільше зчеплення (CBO = 4).

6 Цикломатична складність

N — вершини, E — спрямовані переходи, P — кількість компонент зв’язності без урахування напрямку. Для вузла вибору з k виходами до d додаємо k − 1. Початкові й кінцеві псевдовузли враховано у N. Еталонні графи реконструйовано за інвентарем [1, §3.10]; таблиця показує обидва способи підрахунку.

Діаграма	N	E	P	За графом E − N + 2P	За виборами d + 1
Лаб. 4 Діяльність	22	24	1	24 − 22 + 2 = 4	2 + 1 = 3
Лаб. 5 Стани кур’єра	9	12	1	12 − 9 + 2 = 5	(1 + 2 + 1) + 1 = 5

Лаб 4 Діяльність оформлення і доставки

22 вершини: 15 дій, початок і кінець, fork і join, два вузли рішення та один merge. У 21 некінцевої вершини є щонайменше один вихід. Два рішення й fork мають ще по одному виходу, тому E = 21 + 2 + 1 = 24.

Вибір «Кур’єр прийняв?» утворює цикл повторної пропозиції. «Клієнт на місці?» дає другий вибір. Приготування в закладі й пошук кур’єра запускаються разом: fork додає ребро до структурного графа, але не додає альтернативи виконання.

Лаб 5 Стани кур’єра

Після розгортання стану «На зміні» отримуємо сім простих станів та два псевдостани. Історію замінено ребром «Перерва → Очікує»: перерва починається лише з очікування, тож інших станів повернення немає.

Виходи: «Офлайн» — 2, «Очікує» — 3, «ОтримавПропозицію» — 2. Решта некінцевих вершин мають по одному виходу. Тому E = 8 + 1 + 2 + 1 = 12, а d = 1 + 2 + 1 = 4. Тут немає fork, тому результати однакові.

Що передати до Лаб 10

Модель	Базові сценарії	Додаткова перевірка
Діяльність	Прийняття з першої спроби; після відмови; клієнта немає на місці.	Для join перевірити обидва порядки завершення: спочатку готова їжа / спочатку знайдено кур’єра.
Стани	Завершення з офлайн; вхід/вихід зі зміни; перерва; відмова; прийняття з доставкою.	У кожній послідовності перевірити переходи та очікувані кінцеві стани. П’ять — розмір базису, не повний набір тестів.

7 Стабільність модулів

Ca — кількість різних модулів, які залежать від поточного; Ce — кількість модулів, від яких залежить поточний. I = Ce / (Ca + Ce). Рахуємо прямі залежності Rel між п’ятьма модулями з рисунка 3, включно з підписками на типи подій. Зовнішні системи й транзитивні залежності виключено.

Модуль	Ca Хто залежить	Ce Від кого залежить	I
Сповіщення	0 (—)	4 (Доставка, Замовлення, Оплата, Спільне)	1,00
Доставка	1 (Сповіщення)	2 (Замовлення, Спільне)	0,67
Замовлення	2 (Сповіщення, Доставка)	2 (Оплата, Спільне)	0,50
Оплата	2 (Сповіщення, Замовлення)	1 (Спільне)	0,33
Спільне	4 (Сповіщення, Доставка, Замовлення, Оплата)	0 (—)	0,00

Перевірка. Суми Ca і Ce дорівнюють 9. «Доставка»: Ca = 1 («Сповіщення»), Ce = 2 («Замовлення», «Спільне»), I = 2/3 ≈ 0,67.

ADP. Циклів немає. Топологічний порядок: Сповіщення → Доставка → Замовлення → Оплата → Спільне. Усі стрілки йдуть уперед у цьому порядку.

SDP. Для кожного з 9 ребер перевірено I джерела > I цілі за точними дробами. Порушень немає. I = 1 у «Сповіщень» допустиме; I = 0 у «Спільного» означає, що його зміни можуть зачепити чотири модулі.

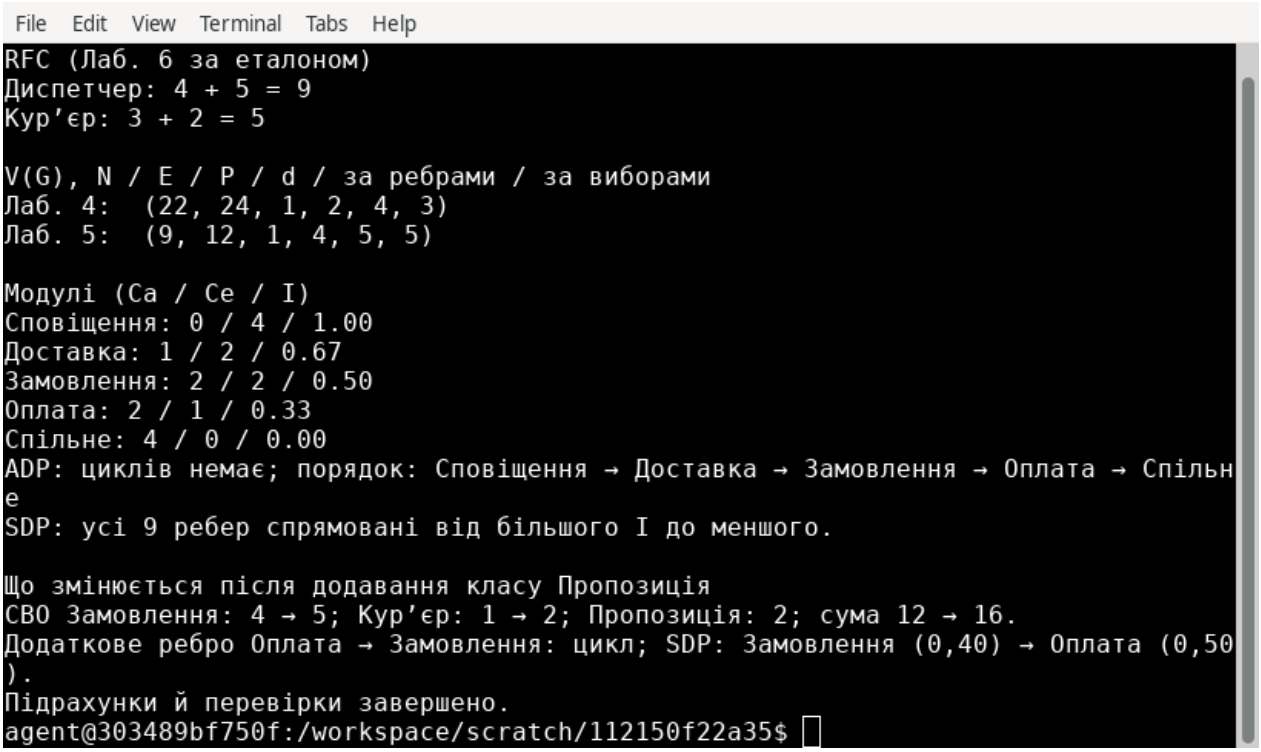


Рисунок 4 — Справжній знімок вікна термінала після запуску перевірки. Показано RFC, V(G), Ca/Ce/I, ADP і SDP та перерахунок після змін.

8 Покращення моделей і повторний підрахунок

Пропозиція	Підстава в метриках	Як перевірити результат
П1 Узгодити операції кур'єра в Лаб. 2 і 6	WMC = 2 у діаграмі класів, M = 3 у RFC. Операція отримання пропозиції є лише в послідовності.	Додати та узгодити назву операції у двох моделях. WMC кур'єра стане 3; RFC лишиться 5 за тим самим сценарієм.
П2 Моделювати «Пропозицію» в «Доставці»	Диспетчер має RFC = 9; у його сценарії створюється пропозиція. Лаб. 8 Р7 фіксує її відсутність у класовій моделі.	Додати клас з двома асоціаціями і зафіксувати правило однієї активної пропозиції. Перерахунок CBO наведено нижче.
П3 Обмежити залежності «Спільного»	Ca = 4, Ce = 0, I = 0,00. Зміни його типів зачіпають усі інші модулі.	Зберігати лише спільні стабільні типи. Тест має відхиляти імпорт «Спільним» будь-якого бізнес-модуля.
П4 Перевіряти ADP і SDP при збиранні	Нині 9 ребер, 0 циклів і 0 порушень SDP. Зворотна залежність оплати змінює ці результати.	На графі перевірено, що ребро «Оплата → Замовлення» створює цикл. Додати аналогічну перевірку імпортів у CI.

Кількісна оцінка пропозиції П2

До шести базових пар додаємо «Пропозиція → Замовлення» і «Пропозиція → Кур'єр». Напрям навігації залишає знання про обидва об'єкти у «Пропозиції», але за правилом курсу CBO рахуємо симетрично.

Клас або показник	До зміни	Після зміни	Зміна
Замовлення	4	5	+1
Кур'єр	1	2	+1
Пропозиція	—	2	новий клас
Сума CBO	**12**	**16**	**+4 = 2 × 2**

Зростання CBO — ціна за явне представлення пропозиції. Залишити цей клас усередині «Доставки»; перехід на ідентифікатори замість об'єктних посилань оцінити окремо, якщо зростання залежностей ускладнить супровід.

У контрольній зміні П4 отримано I(Замовлення) = 2/5 = 0,40 та I(Оплата) = 2/4 = 0,50. Після перерахунку старе ребро «Замовлення → Оплата» порушує SDP, а пара зустрічних ребер порушує ADP. Початкову діаграму не змінено.

9 Протокол використання ШІ

ШІ використано для зіставлення матеріалів, чернетки сценаріїв, порівняння альтернатив і написання програми перевірки. Остаточний вибір і числа спираються на явно записані правила та відтворювані підрахунки.

Запит до ШІ	Що використано	Що перевірено або виправлено
Порівняй три стилі за контекстом пілота	Матриця C1–C4, пріоритет C1 і C2, варіант модульного моноліту.	Мікросервіси не одержали бал за розгортання без бюджету на супровід. Цілі C1–C4 не видано за виміряні.
Перевір класи та графи за правилами курсу	Інвентар CK і RFC; відтворені графи діяльності й станів.	Вручну звірено Замовлення і Кур’єра; Σ CBO = 2 × 6. Fork включено в E, виключено з d.
Звір C4 і перерахуй модульні залежності	Парсинг Rel із .puml, підрахунок вузлів та ребер, контрольна зміна графа.	Звірено три .puml з MD і видимий текст SVG з HTML. Перевірено топологічний порядок та 9 умов SDP.

Для повторення перевірки у папці з трьома парами PUML/SVG виконати команду нижче. Параметр --guide-dir дозволяє додатково звірити MD і HTML, якщо методичка лежить у вказаній папці.

Висновки

Модульний моноліт набрав 16 з 18 балів за сценаріями стартової версії. Три діаграми C4 фіксують межу сервісу, п’ять контейнерів та п’ять серверних модулів. Рішення зменшує складність випуску, але залишає спільну точку відмови процесу, яку потрібно перевірити за C3.

Найвище зчеплення в «Замовлення» (CBO = 4), найбільший RFC серед двох досліджених класів у «Диспетчера» (9). Для діяльності V(G) дорівнює 4 за графом і 3 за виборами, для станів — 5 обома способами. Дев’ять модульних залежностей задовольняють ADP і SDP.

Додатки Код діаграми контексту

```
@startuml
!include <C4/C4_Context>
title Діаграма контексту (C4, рівень 1): сервіс доставки їжі — версія для невеликої команди
LAYOUT_TOP_DOWN()
HIDE_STEREOTYPE()

Person(клієнт, "Клієнт", "Замовляє їжу, відстежує доставку")
Person(працівник, "Працівник закладу", "Приймає замовлення, оновлює меню")
Person(кур’єр, "Кур’єр", "Приймає пропозиції, доставляє замовлення")

System(сервіс, "Сервіс доставки їжі", "Замовлення страв у закладах міста та їх доставка кур’єрами")

System_Ext(платіжний, "Платіжний шлюз", "Обробка банківських карток")
System_Ext(карти, "Картографічний сервіс", "Геокодування та маршрути")
System_Ext(push, "Сервіс push-сповіщень", "Доставка сповіщень на пристрої")

Rel(клієнт, сервіс, "Оформлює й оплачує замовлення")
Rel(працівник, сервіс, "Приймає замовлення, керує меню")
Rel(кур’єр, сервіс, "Отримує пропозиції, підтверджує доставку")
Rel(сервіс, платіжний, "Проводить оплату", "HTTPS")
Rel(сервіс, карти, "Будує маршрути", "HTTPS")
Rel(сервіс, push, "Надсилає сповіщення", "HTTPS")

skinparam legendFontColor #222222
skinparam legendBackgroundColor #FAFAFA
skinparam legendBorderColor #BBBBBB
legend right
  |<#08427B>   | особа (користувач) |
  |<#1168BD>   | наша програмна система |
  |<#999999>   | зовнішня система |
endlegend
@enduml
```

Код діаграми контейнерів

```
@startuml
!include <C4/C4_Container>
title Діаграма контейнерів (C4, рівень 2): модульний моноліт сервісу доставки
LAYOUT_TOP_DOWN()
HIDE_STEREOTYPE()

Person(клієнт, "Клієнт", "Замовляє їжу, відстежує доставку")
Person(працівник, "Працівник закладу", "Приймає замовлення, оновлює меню")
Person(кур'єр, "Кур'єр", "Приймає пропозиції, доставляє замовлення")

Boundary(сервіс, "Сервіс доставки їжі", "програмна система") {
  Container(застКлієнта, "Застосунок клієнта", "Flutter", "Каталог, кошик, оплата, відстеження")
  Container(кабінет, "Кабінет закладу", "React, SPA", "Керування меню та замовленнями")
  Container(застКур'єра, "Застосунок кур'єра", "Flutter", "Пропозиції, маршрут, підтвердження")
  Container(сервер, "Серверний застосунок", "Java 21, Spring Boot, модульний моноліт", "Модулі: замовлення, оплата, доставка, сповіщення;
REST API")
  ContainerDb(бд, "База даних", "PostgreSQL", "Окрема схема для кожного модуля")
}

System_Ext(платіжний, "Платіжний шлюз", "Обробка банківських карток")
System_Ext(карти, "Картографічний сервіс", "Геокодування та маршрути")
System_Ext(push, "Сервіс push-сповіщень", "Доставка сповіщень на пристрої")

Rel(клієнт, застКлієнта, "Користується")
Rel(працівник, кабінет, "Користується")
Rel(кур'єр, застКур'єра, "Користується")
Rel(застКлієнта, сервер, "Викликає API", "HTTPS/JSON")
Rel(кабінет, сервер, "Викликає API", "HTTPS/JSON")
Rel(застКур'єра, сервер, "Викликає API", "HTTPS/JSON")
Rel(сервер, бд, "Читає й записує", "SQL")
Rel(сервер, платіжний, "Проводить оплату", "HTTPS")
Rel(сервер, карти, "Будує маршрути", "HTTPS")
Rel(сервер, push, "Надсилає сповіщення", "HTTPS")

skinparam legendFontColor #222222
skinparam legendBackgroundColor #FAFAFA
skinparam legendBorderColor #BBBBBB
legend right
  |<#08427B>   | особа (користувач) |
  |<#438DD5>   | контейнер |
  |<#999999>   | зовнішня система |
  | - - - | межа програмної системи |
endlegend
@enduml
```

Додаток Г Код діаграми модулів

```
@startuml
!include <C4/C4_Component>
title Діаграма компонентів (C4, рівень 3): модулі серверного застосунку та їхні залежності
LAYOUT_TOP_DOWN()
HIDE_STEREOTYPE()

Boundary(сервер, "Серверний застосунок (модульний моноліт)", "контейнер") {
  Component(сповіщення, "Модуль «Сповіщення»", "Java-пакет", "Слухає події інших модулів, формує сповіщення\nCa = 0, Ce = 4, I = 1,00")
  Component(доставка, "Модуль «Доставка»", "Java-пакет", "Диспетчер, пропозиції, відстеження кур’єрів\nCa = 1, Ce = 2, I = 0,67")
  Component(замовлення, "Модуль «Замовлення»", "Java-пакет", "Кошик, оформлення, статуси замовлень\nCa = 2, Ce = 2, I = 0,50")
  Component(оплата, "Модуль «Оплата»", "Java-пакет", "Проведення оплат і повернень\nCa = 2, Ce = 1, I = 0,33")
  Component(спільне, "Модуль «Спільне»", "Java-пакет", "Ідентифікатори, гроші, базові події\nCa = 4, Ce = 0, I = 0,00")
}

Rel(сповіщення, замовлення, "Слухає події")
Rel(сповіщення, доставка, "Слухає події")
Rel(сповіщення, оплата, "Слухає події")
Rel(доставка, замовлення, "Слухає «ЗамовленняОплачено»")
Rel(замовлення, оплата, "провестиОплату()")
Rel(сповіщення, спільне, "Використовує")
Rel(доставка, спільне, "Використовує")
Rel(замовлення, спільне, "Використовує")
Rel(оплата, спільне, "Використовує")

skinparam legendFontColor #222222
skinparam legendBackgroundColor #FAFAFA
skinparam legendBorderColor #BBBBBB
legend right
  |<#85BBF0>   | компонент (модуль) |
  | - - - | межа контейнера |
  | Ca | доцентрові залежності (хто залежить від модуля) |
  | Ce | відцентрові залежності (від кого залежить модуль) |
  | I | нестабільність Ce / (Ca + Ce) |
endlegend
@enduml
```